Threadplane

Connecting Agents to Applications

Ownership, protocols and recovery across Angular and React.

Threadplane · Open source · Angular + React

Engineering guide · 2026 · Research reviewed November 7

threadplane.ai

Inside this guide

Practical boundaries, current APIs, and decisions to make before you ship.

01 Give the application a precise vocabulary

02 Choose protocols by the boundary they cross

03 Make session lifetime independent of rendering

04 Reduce events into coherent observations

05 Keep commands explicit and presentation local

06 Restore navigation without confusing it with persistence

07 Route decisions through capability-specific commands

08 Dated compatibility and reference notes

For senior engineers and technical leaders

Framework-neutral architecture, with Angular and React examples. Published Angular adapters and native source-build bindings are distinct profiles; each example identifies its scope. Native packages remain private and are not npm installation instructions.

Research and repository API review: November 7, 2026. Model catalogs and upstream specifications are dated observations, not a promise of Threadplane support. Threadplane is free and open source; you own your backend, data and deployment.

Give the application a precise vocabulary

A live agent integration connects several clocks: the user's actions, the model's output, the runtime's execution, and the browser's rendering. A reliable interface must explain the current situation even when those clocks disagree. The most useful abstraction is an application-owned session with explicit commands and observable state. Angular and React can present that state differently while preserving the same business meaning.

Consider a hypothetical invoice investigation. An analyst asks whether invoice INV-1042 duplicates an earlier charge. The assistant retrieves evidence, proposes an illustrative USD 480.00 credit, waits for approval, and asks a finance service to execute it. The user may navigate away, return from another device, or lose connectivity after approval. The integration must distinguish what was requested, what was observed, and what is known to have completed.

Name identities according to their responsibility

Identity	What it identifies	What it does not prove
Thread	A retained conversation or workflow context	Current authorization or permanent retention
Run	One execution attempt within that context	Completion of every external action
Message	A stable transcript item	A distinct network frame or authoritative fact
Tool call	One proposed or executing invocation	Successful business execution
Checkpoint	A runtime's saved execution state	An atomic commit with another service
Decision token	A particular pending proposal and revision	Permission to approve or an execution receipt
Operation	A business action whose outcome can be reconciled	That the browser received its result

In the example, the same thread can contain two investigations. A second run might correct an interpretation from the first. Each run can search several invoices with the same tool name, so tool names cannot serve as call identities. Proposal P-7 can lead to operation OP-23, whose result remains relevant even if the message containing its receipt has not arrived.

Design for disagreement between observations

A transcript may still show “preparing a credit” while the operation service already records success. A connection may be closed while the run is active. The application should preserve these distinctions instead of forcing everything into a single loading boolean.

Define a small domain model before building the screen. It should explain pending selection, active execution, waiting for a decision, checking an outcome, and a terminal result. Not every runtime exposes these concepts in the same shape; the application may compose them from several sources. The protocol supplies evidence, and the application assigns product meaning. AG-UI's specification is an example of an event contract with explicit behavioral requirements, rather than a substitute for the application's business model. Primary reference: AG-UI specification.

Choose protocols by the boundary they cross

The word protocol often hides several unrelated jobs. A model-provider API carries prompts, outputs, and tool requests. An orchestration runtime decides how work proceeds and persists. An application event protocol communicates execution to a client. A rendering protocol describes a surface. An integration may use several of these at once without making any one of them the universal agent API.

Map responsibilities before choosing adapters

Surface	Principal responsibility	Application question
Provider API	Model invocation and supported tools	Which API/model combination has been evaluated?
Runtime API	Execution, state, persistence, resume	What survives interruption, and who owns it?
AG-UI	Ordered agent-to-application events	Which events and command bindings does this adapter support?
MCP	Tool and resource integration	Which server may expose which capability to this user?
A2A	Collaboration between agent systems	How are task ownership and delegated outcomes represented?
A2UI or json-render	Structured interface description	Which version and catalog can this renderer accept?

MCP also has an Apps extension for tool-associated interactive HTML surfaces hosted in a controlled container. It should not be described as having no UI capability. That extension solves a different integration problem from selecting native Angular or React components in an application-owned registry. Primary references: MCP specification and MCP Apps overview.

A2A addresses communication between agents, including task-oriented interactions. It does not automatically define the invoice workspace's transcript, draft, or approval card. If a specialist agent returns a finding, the application still needs to preserve its provenance and decide whether the main investigator may rely on it. Primary reference: A2A specification.

Specify the missing seams

A protocol label alone does not establish authentication, resume semantics, or cancellation. Write down the concrete binding: transport, ordering, identity fields, supported events, terminal conditions, and recovery commands. Include what happens when an event is unknown or the connection ends before the expected terminal signal.

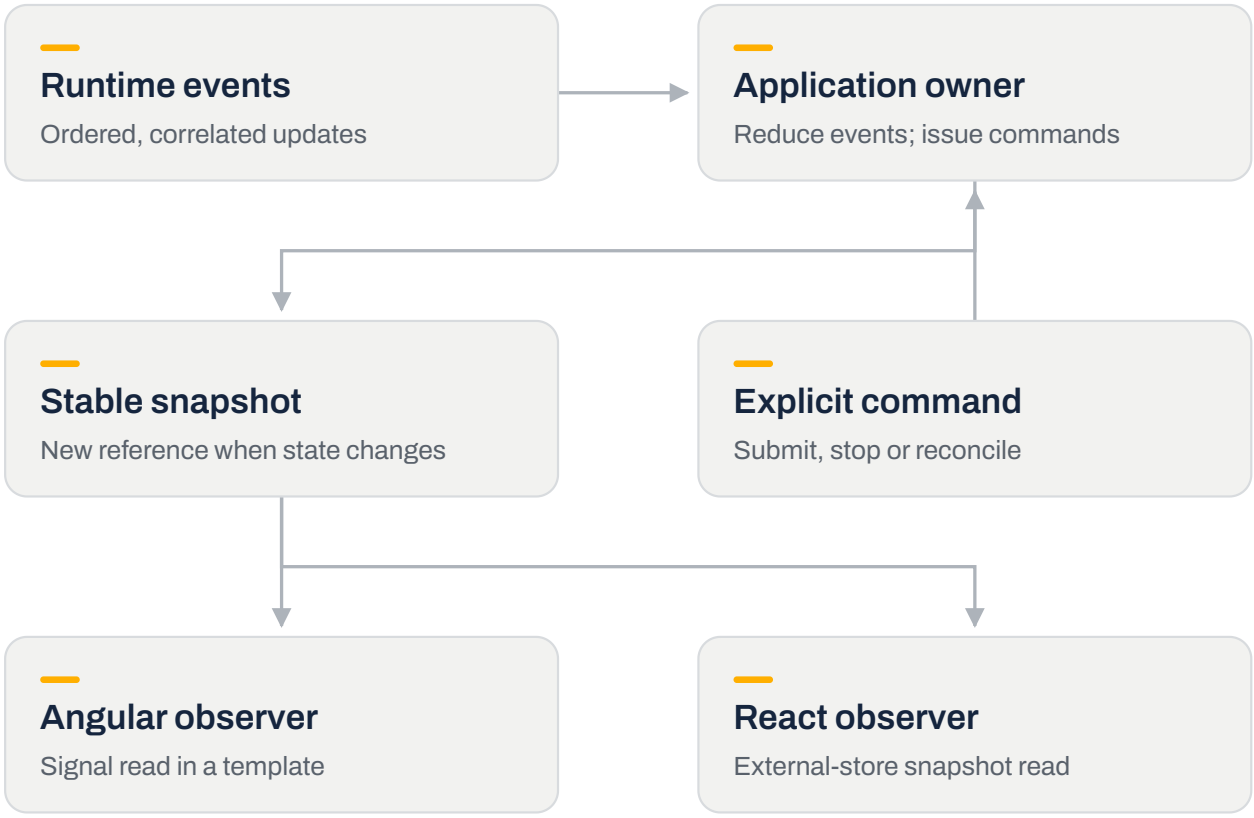
For the invoice workflow, an AG-UI stream might carry investigation progress while a separate application endpoint accepts the analyst's decision. A rendering payload might describe an evidence comparison. The action service still owns OP-23. Combining those contracts is legitimate; pretending they are one interchangeable interface makes recovery harder.

Prefer a deterministic application endpoint when the operation is already fully specified. Loading a known invoice by identifier does not need a model loop. This is also a useful baseline for diagnosing problems: verify ordinary authorization and data loading before introducing agent execution. Consult the selected protocol's own transport contract instead of assuming every streaming endpoint behaves like another. Primary reference: AG-UI transports.

Make session lifetime independent of rendering

The application should decide when a session begins, which user and thread it represents, and when its resources are released. Components borrow that owner to read snapshots. A toolbar, transcript, and status badge can observe the same session without creating three runs or three competing copies of state.

One owner, multiple observers



The application owns execution and disposal. Framework observers read the same owner and release only their subscriptions. Rendering and remounting do not submit work.

In the example, the workspace owner remains active while the analyst opens an evidence drawer. Closing the drawer releases its subscription. Switching accounts requires a deliberate application transition that clears or replaces the relevant owner. Neither event should be inferred from the presence of a particular DOM element.

Angular: observe in the injection context

The native Angular observer requires an injection context and returns a readonly signal. The following source-build example receives an already-created session through an injection token. It only displays status. The surrounding application must provide that session and own its disposal.

```
import { Component, inject, InjectionToken } from '@angular/core';
import { observeAgent } from '@threadplane/angular';
import type { AgentSession } from '@threadplane/core';

// Source-build profile. The application provides an existing owner.
export const SESSION = new InjectionToken<AgentSession>('session');

@Component({
  selector: 'app-agent-status',
  standalone: true,
  template: `<p>Run status: {{ snapshot().status }}</p>`,
})
export class AgentStatus {
  readonly session = inject(SESSION);
  readonly snapshot = observeAgent(this.session);
  // Destroying this view releases its subscription, not the session.
}
```

Use local signals for presentation state such as a draft or an expanded evidence row. Use computed values for derived views rather than copying the session's messages into another writable collection. Signals offer the reactive mechanism; the ownership contract comes from the application and Threadplane's observer implementation. Primary reference: Angular signals.

React: subscribe to an existing owner

The corresponding native React example receives the same kind of borrowed owner as a prop. It returns the current snapshot through `useAgent`; it does not start a run, submit a message, or stop the session.

```
import { useAgent } from '@threadplane/react';
import type { AgentSession } from '@threadplane/core';

// Source-build profile. The application passes an existing owner.
export function AgentStatus({ session }: { session: AgentSession }) {
  const snapshot = useAgent(session);
  return <p>Run status: {snapshot.status}</p>;
  // Unmounting releases this subscription, not the session.
}
```

React's external-store contract requires a stable snapshot between changes. Threadplane's native hook delegates to that mechanism and releases the subscription on unmount. The current implementation does not supply a server snapshot, so this example is a client integration and makes no SSR or hydration claim. Primary reference: React useSyncExternalStore.

Create effectful application owners outside React rendering under an explicit lifetime policy. A lazy state initializer is not a safe hiding place for a constructor that starts sockets or work: development StrictMode can call initializers more than once. Mounting a view should remain an observation, and a user command should remain a command. Primary reference: React StrictMode.

Keep published and native profiles distinct

Published Angular uses provider-based adapter APIs such as `provideAgent` and `injectAgent`. Native Angular and React use the borrowed-owner pattern shown here and currently require the repository's source-build setup. Even similarly named methods may accept different arguments. The Angular introduction describes the published integration; the React introduction explains the native React boundary. Neither is a substitute for the native Angular build instructions.

For the native examples, use the repository setup guide at the researched commit and the native Angular project targets. That profile uses Node 22 and npm 10, root `npm ci`, then `npx nx build native-conversation-angular --configuration=production` or the corresponding React target. The tooling builds local candidate tarballs and installs isolated consumers using the root lockfile; it compiles against those installed declarations and runtimes. It does not obtain the private native bindings through a public npm installation.

Serving requires `NATIVE_LANGGRAPH_URL` and `NATIVE_ASSISTANT_ID`; an optional API key stays in the server-side proxy environment. The native Angular consumer installs core, content, Angular, and the candidate LangGraph package. That candidate exposes the native runtime entry point, unlike the ordinary published LangGraph root. Build or serve through the documented targets so imports resolve to the intended profile. The compact observer above still expects the application to supply an existing session through its token; the setup guide demonstrates the surrounding owner and deployment configuration.

Reduce events into coherent observations

A stream is an ordered account of changes, not a ready-made component tree. The adapter or application reducer should turn accepted events into stable snapshots. Components then render the snapshot appropriate to their task. This keeps transport framing, partial argument handling, and run bookkeeping out of individual cards.

Preserve identity during incremental updates

An assistant message can grow through many fragments while retaining one identity. A tool call can accumulate argument text before it becomes a complete request. Two searches with the same tool name can overlap. Store and display these entities by their actual identities, not their array positions or current text.

Suppose search call A concerns INV-1042 and call B concerns the purchase order. B may finish first. The transcript should attach each result to its corresponding call and preserve meaningful order without claiming A has completed. A component keyed only by tool name can accidentally overwrite one result with the other.

Partial arguments are observations. A tool view may show that an invoice search is being prepared, but should not parse a half-written amount into an actionable proposal. A complete tool result is another observation with its own validation needs. Neither becomes a credit receipt merely because the model describes it confidently.

Publish a consistent aggregate

When several fields describe one transition, update them coherently. A completed run should not briefly appear as still running with an unrelated error from a previous attempt. Keep error association, run identity, and message updates within a defined reduction step. Preserve the last accepted state when a malformed frame is rejected, and expose an appropriate transport or validation problem separately.

For high-frequency output, measure rendering cost before optimizing. Coalescing visible text updates may be appropriate; dropping terminal events or decision changes is not. Limit retained raw frames if they are only needed for bounded diagnostics. The snapshot should remain useful even when no component observed every intermediate fragment.

Treat the end of a connection as evidence, not a verdict

An expected terminal event can establish a protocol-level outcome. A socket closing without it may establish only that observation ended. Do not synthesize completion merely because no new text arrived for a while. Record that the stream was truncated and let the recovery policy determine whether to reconnect, check status, or surface an unresolved condition.

These design choices should be tested against the actual adapter. The Angular streaming guide and React streaming guide document different implementation surfaces. Upstream event patterns provide additional context, but do not certify a custom reducer. Primary reference: AG-UI patterns.

Keep commands explicit and presentation local

A composer has local state: the text being edited, an attachment selection, or a validation message. A session has execution state: the accepted request, active run, and observed result. Mixing those lifetimes causes familiar failures, such as a streamed update clearing a new draft or a rerender submitting work again.

Make submission a deliberate transition

The user presses Send. The application validates the draft, captures the intended thread and request, and invokes the owner. It tracks whether that submission is being accepted and determines when to clear the draft. If submission is rejected before acceptance, the user should retain useful work. If it is accepted but the result is uncertain, restoring the draft and encouraging another Send can create duplicate work.

The native minimal core session accepts text through `submit`, while published Angular adapters expose a different submission contract. Backend-specific sessions may add options. The status examples above intentionally do not hide this distinction behind a generic submit wrapper. Select the exact profile, then implement the command against its real type.

Use an event handler for this user intent. Angular template actions and React event callbacks can both delegate to an application service. Rendering should derive whether Send is available; it should not become the mechanism that sends. Reactive read helpers are valuable for loading data, but their cancellation and rerun behavior should not be casually adopted as the ownership policy for mutations. Primary reference: Angular httpResource guidance.

These paired source-build composers demonstrate one stable borrowed owner. The Angular example uses the injection token from the earlier status example; React receives its owner as a prop. Each keeps the draft local, blocks repeated submission while pending, and clears the draft only for a successful completion outcome.

```
import { Component, inject, signal } from '@angular/core';
import { observeAgent } from '@threadplane/angular';
import { SESSION } from './session-angular';

@Component({
  selector: 'app-agent-composer',
  standalone: true,
  template: `
    <form (submit)="send($event)">
      <label for="request">Investigation request</label>
      <input #field id="request" [value]="draft()"
        [disabled]="pending()" (input)="draft.set(field.value)" />
      <button [disabled]="pending() || needsReview() || snapshot().status ===
'running'">
        Send
      </button>
      <p role="status">{{ notice() }}</p>
    </form>
  `,
})
export class AgentComposer {
  readonly session = inject(SESSION);
  readonly snapshot = observeAgent(this.session);
  readonly draft = signal('');
  readonly pending = signal(false);
  readonly notice = signal('');
  readonly needsReview = signal(false);

  async send(event: Event) {
    event.preventDefault();
    const text = this.draft().trim();
    if (!text || this.pending() || this.needsReview() || this.snapshot().status ===
'running') return;
    this.pending.set(true);
    try {
      const outcome = await this.session.submit(text);
      if (outcome === 'success') this.draft.set('');
      this.needsReview.set(outcome !== 'success');
      this.notice.set(`Run outcome: ${outcome}`);
    } catch {
      this.needsReview.set(true);
      this.notice.set('Check the outcome before submitting again.');
    } finally {
      this.pending.set(false);
    }
  }
}
```

```
import { useId, useRef, useState } from 'react';
import { useAgent } from '@threadplane/react';
import type { AgentSession } from '@threadplane/core';

// Keep this owner stable for the lifetime of this illustrative composer.
export function AgentComposer({ session }: { session: AgentSession }) {
  const snapshot = useAgent(session);
  const id = useId();
  const [draft, setDraft] = useState('');
  const [pending, setPending] = useState(false);
  const [notice, setNotice] = useState('');
  const [needsReview, setNeedsReview] = useState(false);
  const sending = useRef(false);
  async function send(event: React.FormEvent) {
    event.preventDefault();
    const text = draft.trim();
    if (!text || sending.current || needsReview || snapshot.status === 'running')
return;
    sending.current = true;
    setPending(true);
    try {
      const outcome = await session.submit(text);
      if (outcome === 'success') setDraft('');
      setNeedsReview(outcome !== 'success');
      setNotice(`Run outcome: ${outcome}`);
    } catch {
      setNeedsReview(true);
      setNotice('Check the outcome before submitting again.');
    } finally {
      sending.current = false;
      setPending(false);
    }
  }
  return <form onSubmit={send}>
    <label htmlFor={id}>Investigation request</label>
    <input id={id} value={draft} disabled={pending}
      onChange={(event) => setDraft(event.target.value)} />
    <button disabled={pending || needsReview || snapshot.status ===
'running'}>Send</button>
    <p role="status">{notice}</p>
  </form>;
}
```

A non-success outcome or rejected promise closes the example's submission gate. The surrounding application must reconcile uncertain work or handle a pending approval before permitting another submission. Remounting the composer to reset its local flag is not a recovery strategy. These extracts demonstrate deliberate command handling, not a complete multi-thread coordinator; the backend remains authoritative about whether work was accepted or completed.

Project views without duplicating authority

A thread filter can be an Angular signal or React state value. A visible message list can be a pure projection of the current snapshot and filter. The client should not maintain a second writable “agent state” merely to adapt the shape to a component. Every copy creates a synchronization question during reconnect or selection changes.

A useful distinction is between a local view preference and a business decision. Expanding invoice line items belongs to the former. Selecting an invoice as the target of a credit belongs to the latter once it affects the proposal. The application must then bind the selection to the proposal revision and server validation.

Keep controls attached to the appropriate owner. A sidebar can show thread summaries from a directory while the transcript observes one selected session. Clicking Stop should address that selected run, not whichever session happened to be captured by an old callback. Test a selection change during an in-flight request. React's guidance on separating events from effects supports this discipline, while the final command contract remains application-specific. Primary reference: React, Separating Events from Effects.

Restore navigation without confusing it with persistence

A URL can remember which conversation the user wants. It cannot preserve the conversation's server state. A thread directory can list accessible conversations. It cannot guarantee that the selected thread still exists. A local cache can make navigation faster. It cannot establish current permission to read an invoice.

Define the selection transaction

When the analyst selects another investigation, the application should record the selection intent, load or attach the authorized session, and publish the resulting state only if that intent is still current. Late results from the previously selected thread must not overwrite the new selection.

Keep loading, unavailable, denied, and empty states distinguishable. An empty conversation offers a place to begin. A denied conversation should not reveal sensitive metadata. A deleted or expired conversation may need an explanation and a route back to the directory. Silently presenting all three as a fresh blank transcript loses useful context.

Search labels also matter. Filtering titles already loaded in the browser is not a search across all historical conversations. Name and test that boundary. If the product offers server search, preserve its authorization, pagination, and query semantics separately from a local filter.

Treat history as evidence with a scope

A checkpoint list, a message archive, and a branch tree are different views. A loaded history page may be incomplete and need explicit pagination. A historical snapshot may be useful for inspection without being safe to resume. Branching or replay should appear only when the selected runtime and application implement those operations.

In the invoice case, reopening an earlier proposal should make its historical status clear. The analyst must not approve an obsolete copy after a newer proposal has replaced it. A saved generated interface likewise needs a current action policy; its old button labels do not confer present authority.

Reconcile before deciding what to repeat

If OP-23 may have committed, query its status before offering a new action. The application can restore the conversation and independently reconcile the business operation. It should preserve the association even if the final assistant message is missing. The distinction follows from the architecture of durable runtime state and external side effects, not from either frontend framework.

Implementation references: Angular persistence and React persistence. Runtime foundations: LangGraph persistence.

Route decisions through capability-specific commands

Approval is a domain transition with an identity, a version, and an authority check. An interrupt is one possible runtime mechanism for pausing execution. A button click is an expression of user intent. These concepts often meet, but they should not be treated as the same object.

Bind the decision to the proposal

The invoice application can expose a decision token for proposal P-7 and a flag indicating whether the current view may respond. The token identifies the proposal the user reviewed. The server still verifies identity, permission, expiry, revision, and whether a decision already exists. Disabling a button helps prevent accidental duplicate interaction; it does not enforce those rules.

A minimal core session does not include a universal resume method. The native LangGraph profile has a concrete resume capability, while published Angular and application-owned wrappers use different command shapes. The application should map its explicit approve or decline intent into the selected backend contract and retain the decision's status while that request is in flight.

For a connection failure after approval, show that the decision or operation requires checking. Do not silently turn a retry button into a second approval. If another analyst already declined P-7, update the view with that authoritative result. If the proposal expired, fetch a new proposal and require a new decision on its contents.

Verify the boundaries in isolation and together

Test boundary	Essential acceptance evidence
Observer	Mount and unmount change subscriptions without starting or disposing work
Event reduction	Interleaved calls and truncated streams preserve correct identities and uncertainty
Composer	One accepted intent produces one application command; rejected input remains editable
Selection	Late updates cannot replace the currently selected investigation
Decision	Stale and duplicate tokens are rejected or resolved to their established outcome
Recovery	Status checks do not replay an uncertain credit operation

Use deterministic fixtures for Angular and React presentation states, then test the real adapter's command translation and the backend's durable decision flow. A mock that displays “approved” proves only that the view can display that state. An integration test should demonstrate that no protected operation occurs before a valid decision and that a lost response can be reconciled afterward.

The local Angular interrupt guide and React interrupt guide are the relevant implementation references. Primary runtime reference: LangGraph interrupts.

Dated compatibility and reference notes

Checked on 7 November 2026. The examples illustrate real repository profiles rather than universal adapter parity. Native core, Angular, and React packages are private at 0.0.0; the published-line Angular packages are 0.3.7. The native observers borrow their owner and release only subscriptions. React's current observer has no server snapshot.

Upstream and local support are separate facts

Area	Upstream research baseline	Repository baseline
Angular	22 active	Native test baseline 21.1.6; peer range includes 20–22
React	19.3	Native runtime baseline 19.2.4
AG-UI	Specification 1.0	Client/core dependency range ^1.0.1
A2UI	Published specification v1.0	Local wire constant v0.9
json-render	Changelog 0.22.0	Local dependency 0.16.0
MCP / A2A	MCP 2026-07-28; A2A 1.0.0	Protocol discussion is not a blanket implementation claim

A peer range expresses accepted versions, not a completed compatibility matrix. A newer upstream feature should not be inferred from a local dependency on an earlier version. Review changes at the concrete adapter and rendering boundary before upgrading.

Chapter references point to primary contracts and framework guidance. Additional version sources: Angular releases, React versions, A2UI specification, and json-render changelog. The companion From Prototype to Production develops model evaluation and operational ownership; Generative Interfaces develops the rendering boundary.